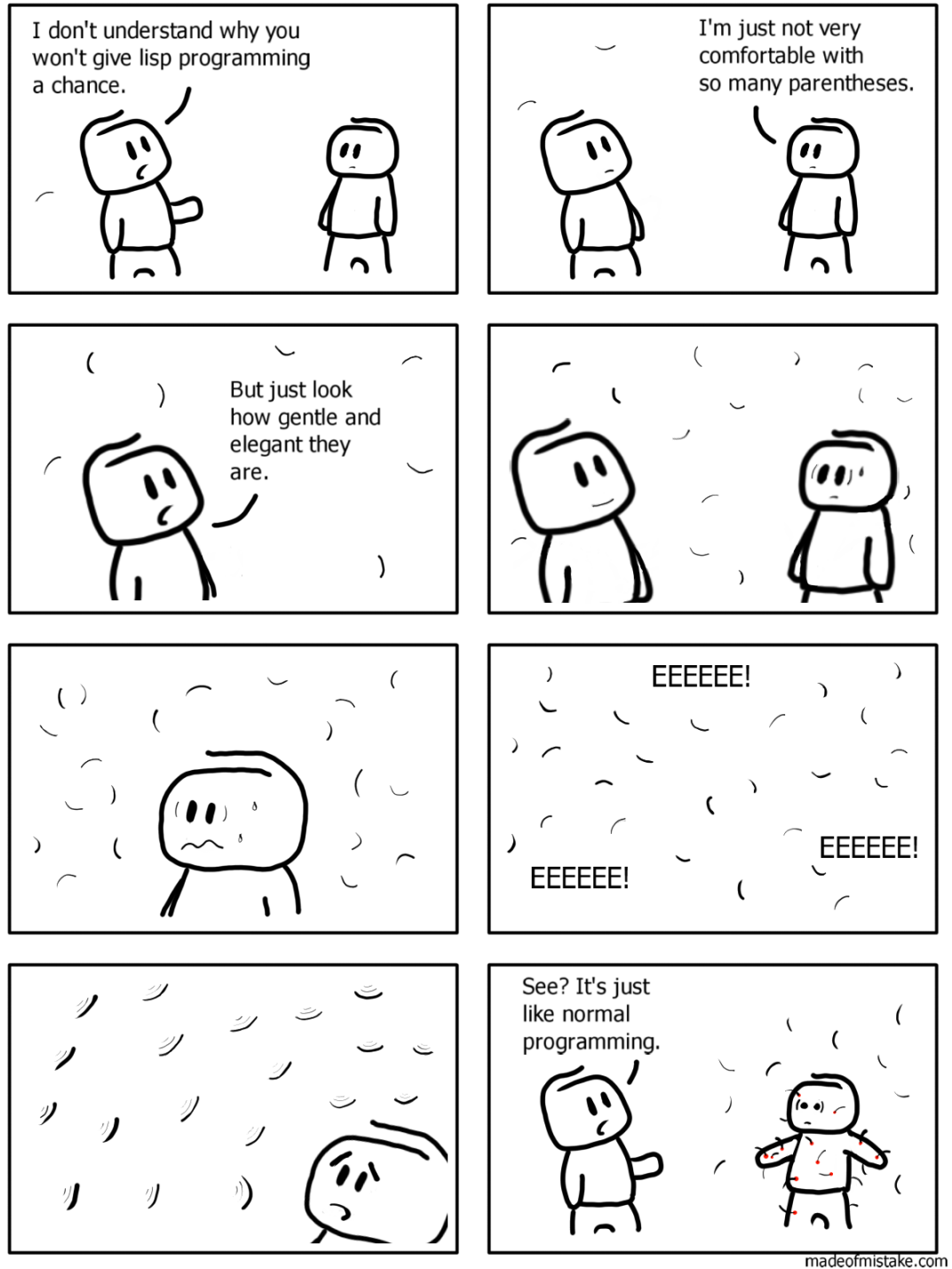
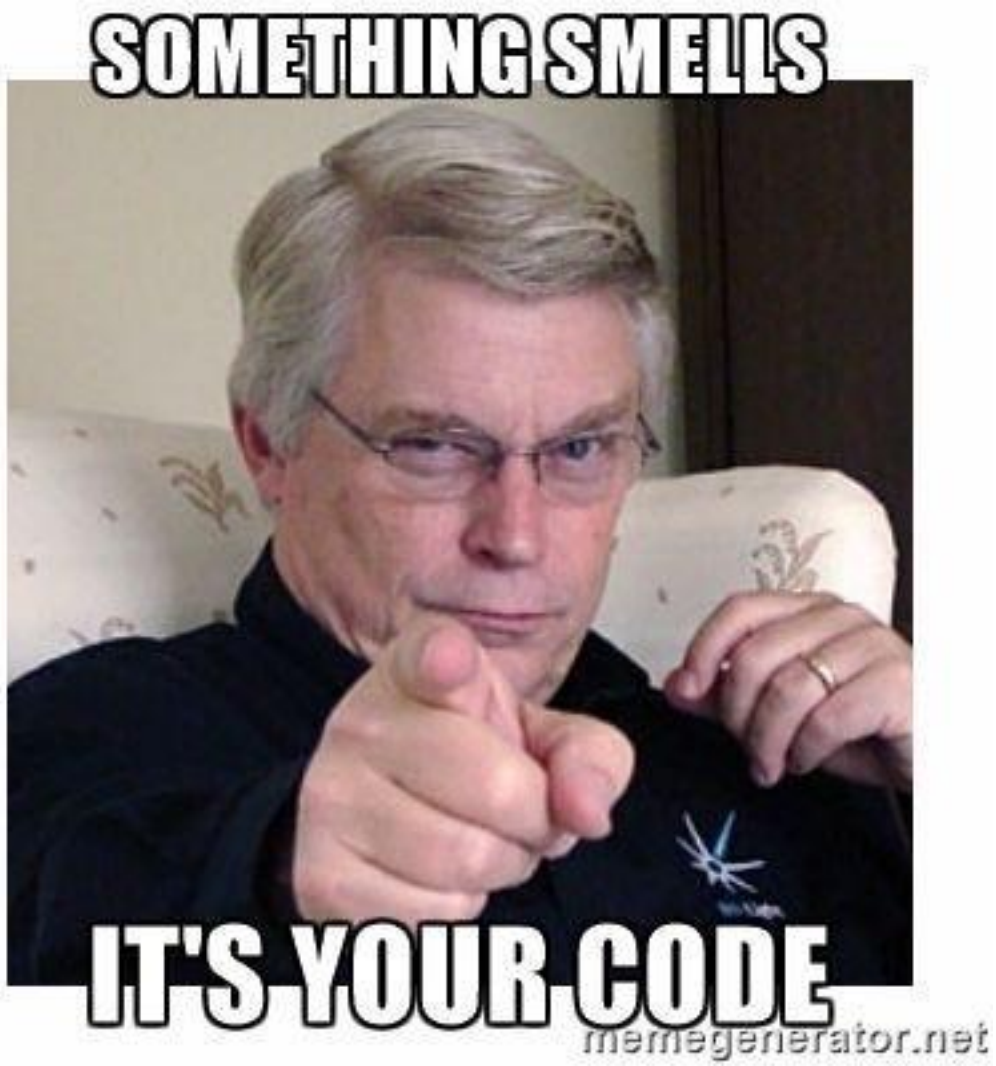

I don't understand why you won't give lisp programming a chance.
I'm just not very comfortable with so many parentheses.
But just look how gentle and elegant they are.
EEEEEE!
EEEEEE!
EEEEEE!
See? It's just like normal programming.
madeofmistake.com

Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S



CSC3003S ASD11
SOLID Design principles (OOP)

- **S**ingle Responsibility
- **O**pen Closed
- **L**iskov substitution
- **I**nterface Segregation
- **D**ependency Inversion

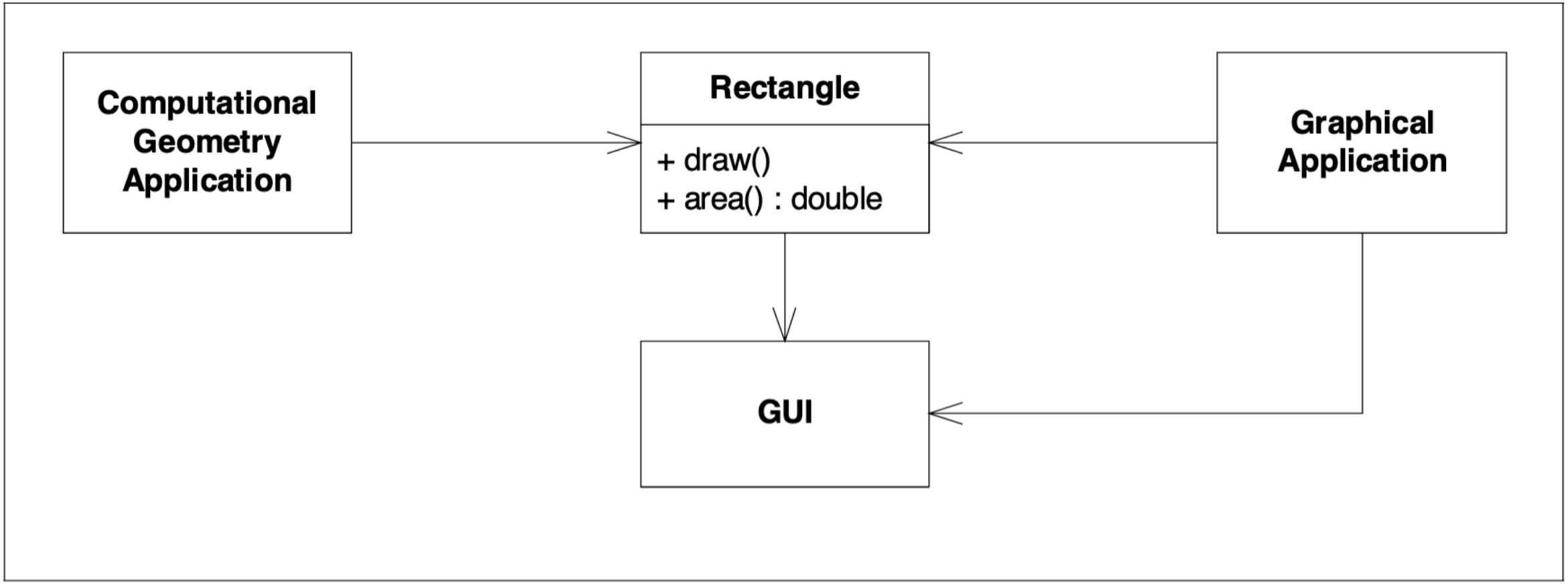
Single responsibility principle (SRP)

Class/function must do one thing and do it well

"There should never be more than one reason for a class [function/module] to change."

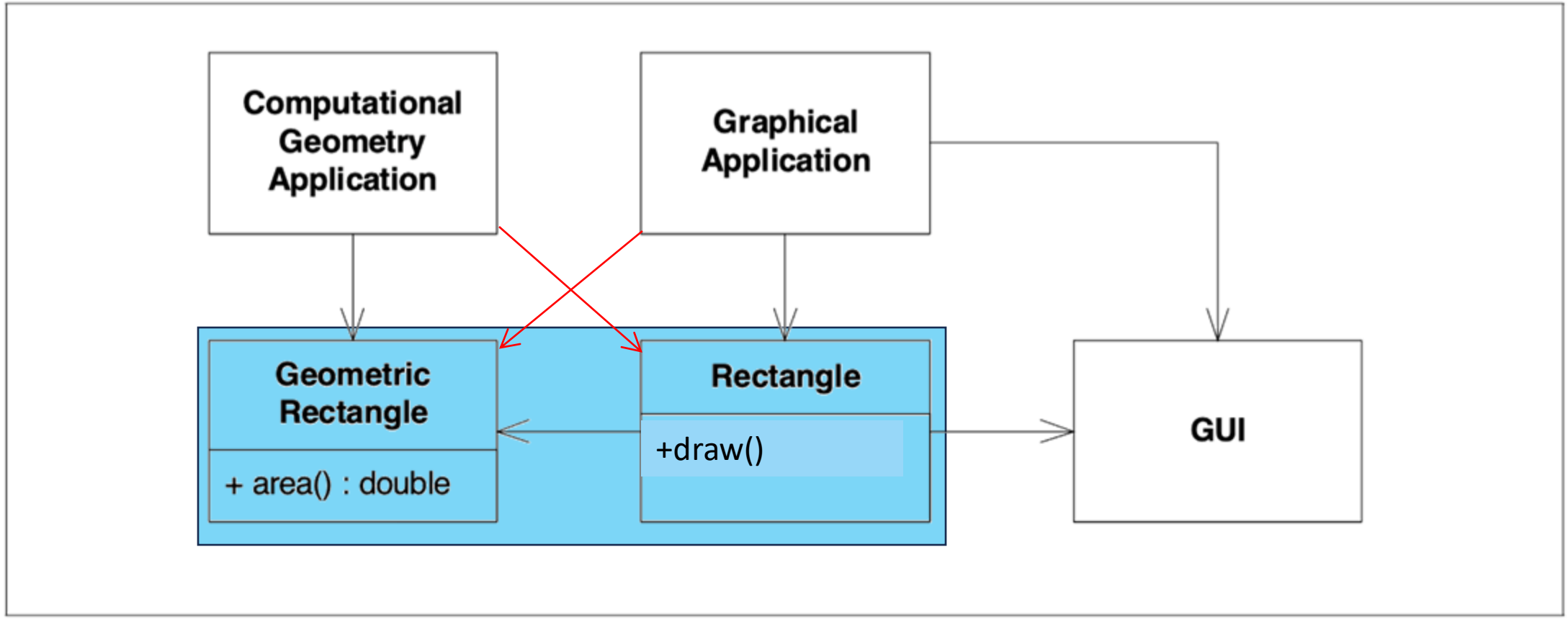
R. Martin, The Principles of OOD, ~1998

Single responsibility violation



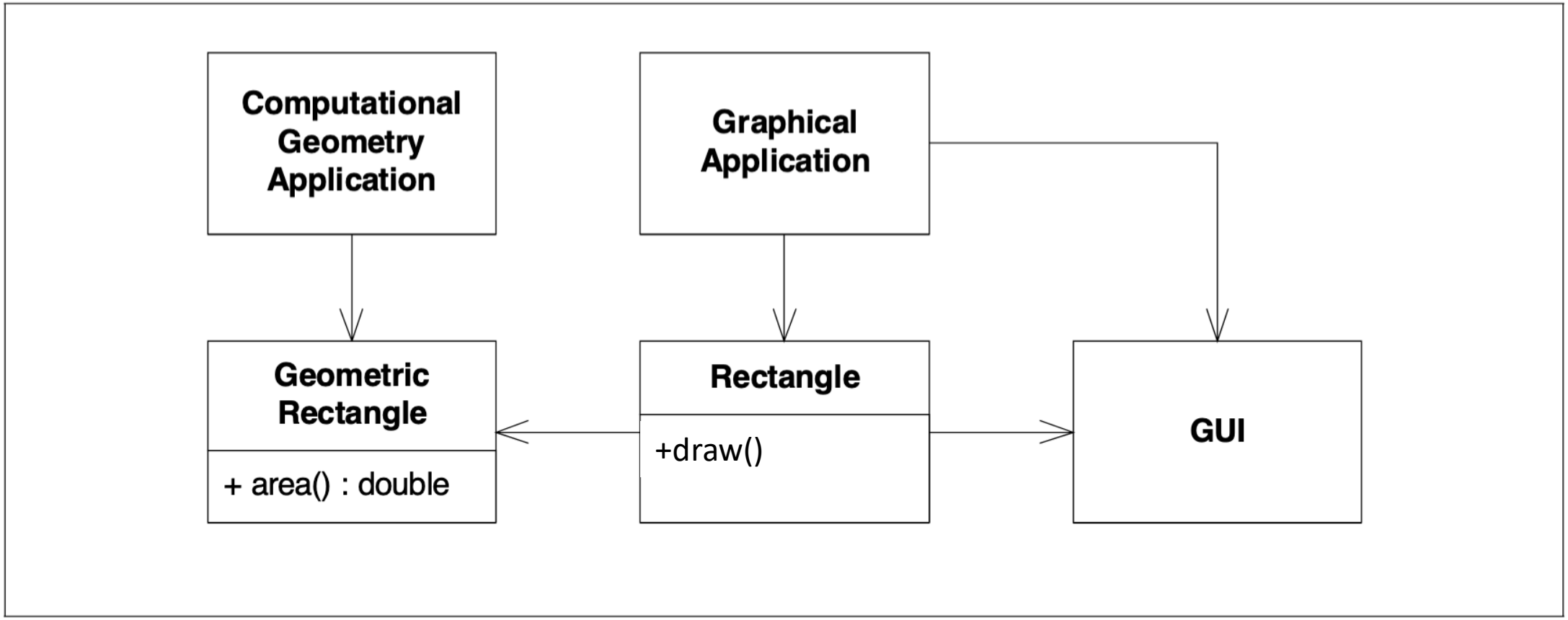
- Rectangle does both drawing and area calculation
- Entangles Geometry & GraphApp
- Changes to draw code and mess up area calc.

What's really happening



- Rect is a compound object
- Hidden cross link between GraphApp & GeometricRect
- High fan out

Single responsibility fixed



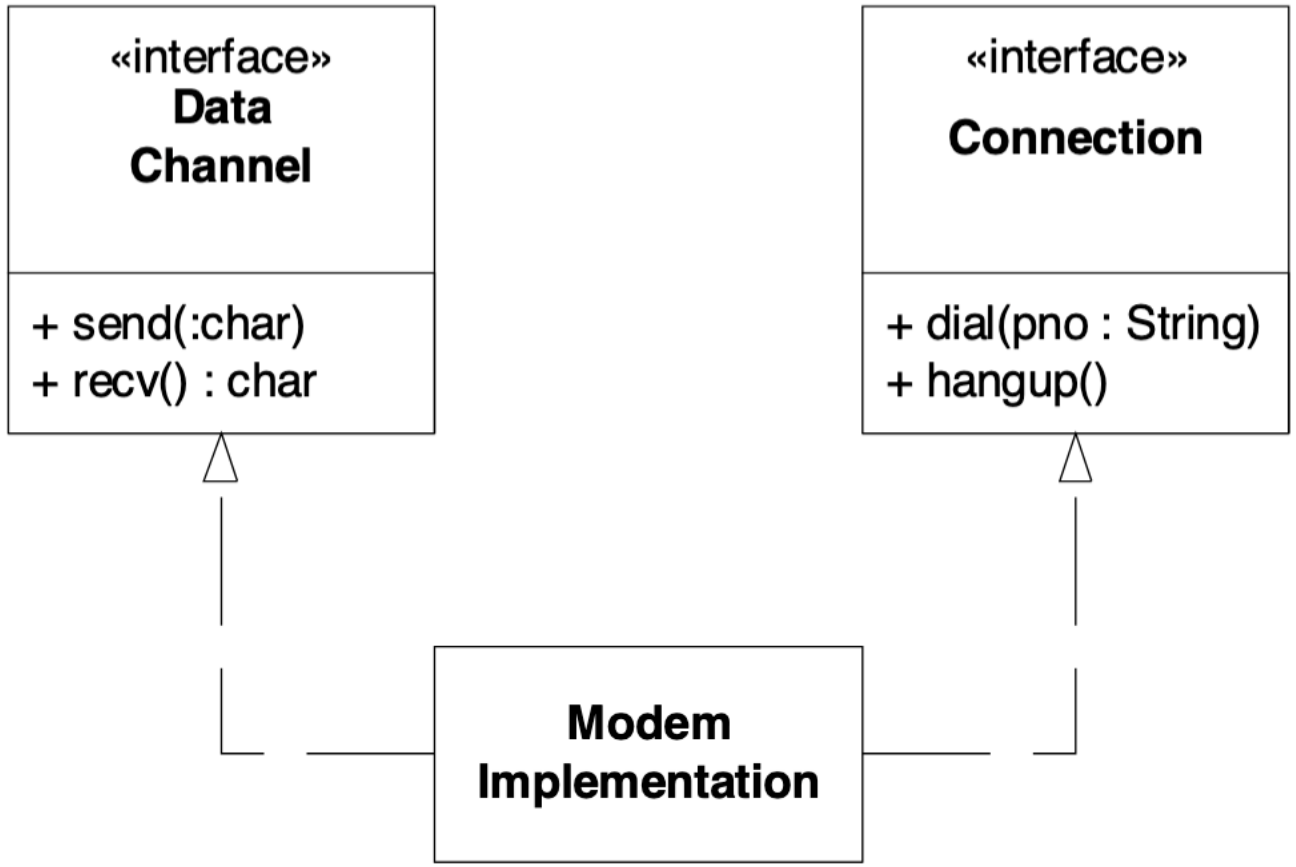
- Hidden cross link between GraphApp & GeometricRect removed
- Low fanout
- Changes to draw code cannot affect Geometry

What's a responsibility?

- A reason for change
- If 2 reasons for a class to change → SRP violation
- Pitfall: Humans cluster responsibilities in groups

```
interface Modem {
    public void dial(String phonenr);
    public void hangup();
    public void send(char c);
    public char receive();
};
```

Separate responsibilities



Connection control: determines the lifetime of the connection

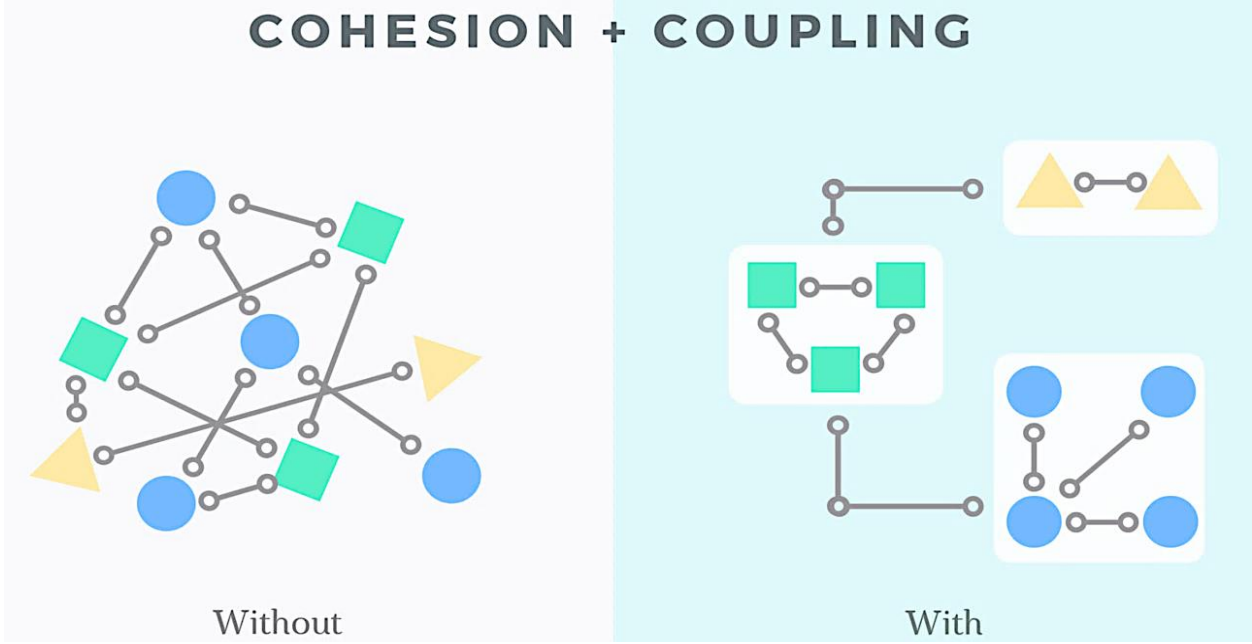
Data control: sends and receives chars

SRP

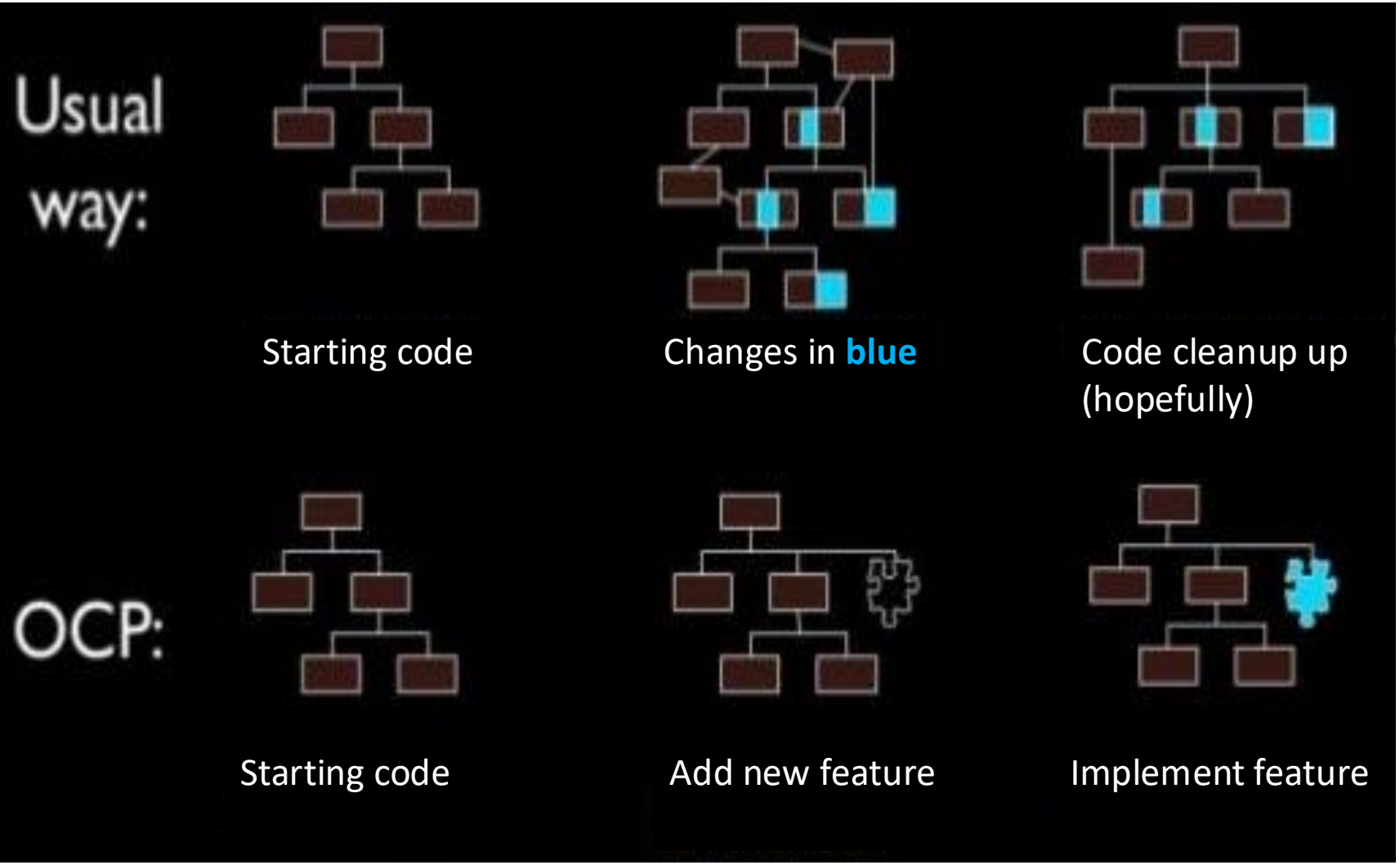
Really a collation of 4 different –older – principles

- Decomposition: Parnas, *On the Criteria To Be Used in Decomposing Systems into Modules*, 1972
- The Separation of Concerns: Dijkstra, *On the role of scientific thought*, 1974
- Coupling and Cohesion: Constantine, *Structured Design*, 1975

Gather together things that change for the same reasons.
Separate things that change for different reasons



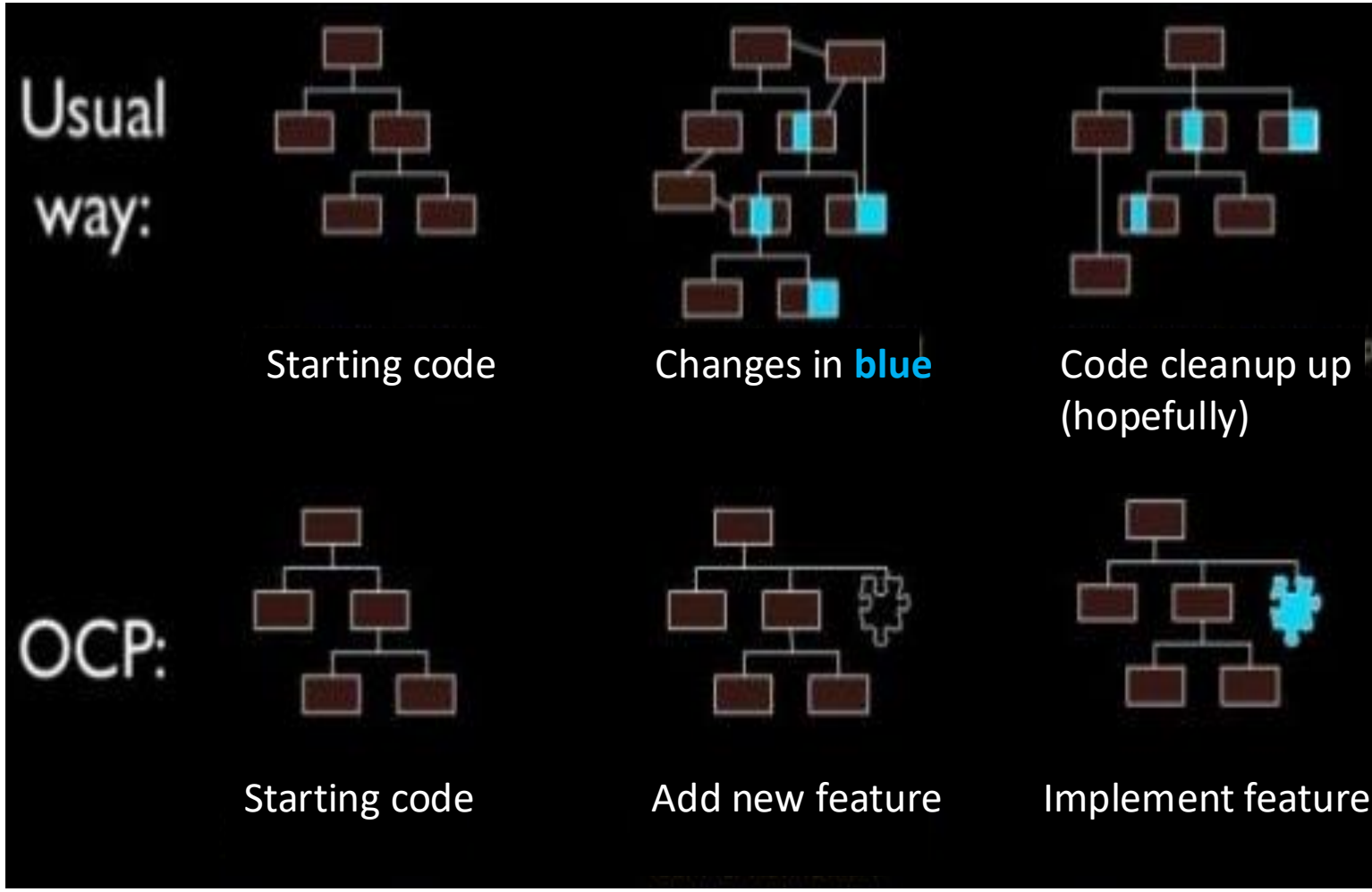
CSC3003S ASD11
SOLID Design principles (OOP)



- **S**ingle Responsibility
- **O**pen Closed
- **L**iskov substitution
- **I**nterface Segregation
- **D**ependency Inversion

Open Closed Principle

"[classes, modules, functions] should be open for extension,
but closed for modification"



Meyer, *Object-Oriented Software Construction*, 1988
https://web.archive.org/web/20060822033314/http://www.objectmentor.com/resources/articles/ocp.pdf

Extension vs modification

- They are “Open For Extension”.
  - New module can extended old module
    - class inheritance (non-virtual: (impossible in Java))
    - interface inheritance (virtual methods: Java)
    - message handlers (Smalltalk, objective C, Windows API)
- They are “Closed for Modification”.
  - No source code changes to old module
  - i.e.: no new features in old source code
    - Bug fixes != modification

Extension options

- class inheritance (non-virtual) C++

```
class Parent {
  void hi(){ printf("hi"); }
}
class Child: private Parent {
  void hi(){ Parent::hi(); printf("C\n"); }
} //warning child method hides parent
```

Interface inheritance: virtual

- polymorphic inheritance (virtual methods: Java)

```
class Parent { virtual void hi(){ ... }}
class Child: private Parent {
  override void hi() {
    Parent::hi(); printf("Child\n"); }
}; //polymorphic inheritance is an antipattern
```

- pure interface

```
class IGreet { virtual void hi() = 0; } //abstract
class Child: public IGreet {
  override void hi() { printf("hi Child\n"); }
}; //better to user interfaces
```

C++ can also do compile-time polymorphism

--out-of-scope-- for ASD

CRTP (Mix-in in rust)

deducing `this` in C++23

Not operator or function overloading!
that is not OOP polymorphism

Message passing: WinAPI, SmallTalk, ObjectiveC

```
const WM_MouseMove := 200; //message id
record TWMMouseMove; begin //message data
  MsgId: int := WM_MouseMove;
  keys: set of [ctrl, alt, shift, caps, win, menu];
  buttons: set of [bL, bR, bM, b4, b5, b6, b7, b8];
  pos: TPoint;
end;
static_assert(sizeof(TWMMouseMove) == sizeof(TWMMsg));
//message handler
function TWindow.MouseMove(Msg: TWMMouseMove) message WM_MouseMove: bool;
begin
  result := (bL in Msg.buttons) and (Msg.keys = []);
  if result then begin
    DrawLine(prev, Msg.pos);
    prev := Msg.pos;
  end;
end; //result = false? call parent's handler

SendMessage(Window1, MouseData); //WinAPI sends this when mouse moves
```



CSC3003S ASD11
SOLID Design principles (OOP)

- **S**ingle Responsibility
- **O**pen Closed
- **L**iskov substitution
- **I**nterface Segregation
- **D**ependency Inversion

Liskov substitution principle

B. Liskov, 1988

What is wanted [is] the following substitution property:
If for each object o_1 of type *S* there is an object o_2 of type *T* such that for all programs *P* defined in terms of *T*, the behavior of *P* is unchanged when o_1 is substituted for o_2 then *S* is a subtype of *T*

"Functions that use references to base classes must be able to use derived objects without knowing it".

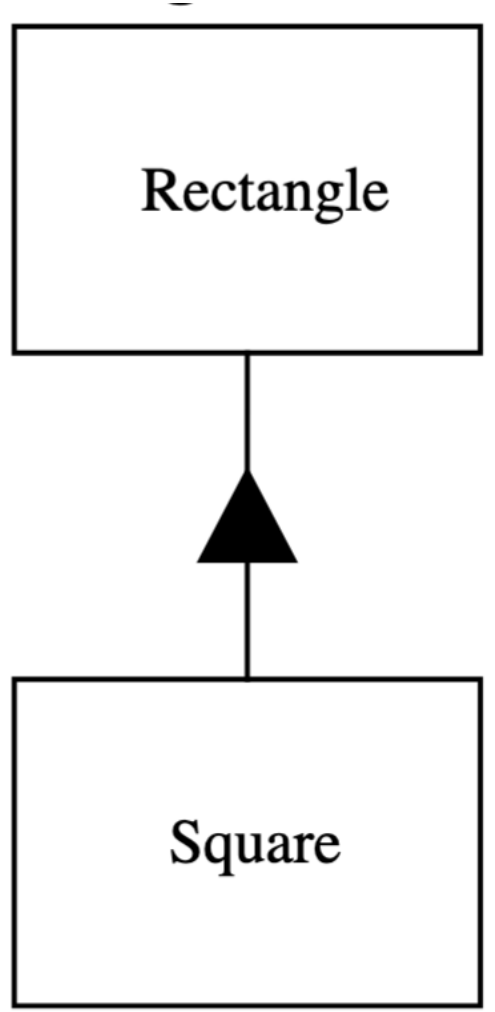
Blatant LSP violation

```
//intentionally bad code
void DrawShape(Shape& s) {          //cast s => Rect
  if (typeid(s) == typeid(Rect)) DrawR((Rect&)s);
  else if (typeid(s) == typeid(Circ)) DrawC((Circ&)s);
}
```

- DrawShape must know all the details of every Shape
- Breaks if new types are added
- Breaks if RTTI is disabled

Subtle LSP violation

```
struct Rectangle {
  void setWidth(const int w) { width = w; }
  void setHeight(const int h) { height = h; }
  double getHeight() const { return height; }
  double getWidth() const { return width; }
private:
  int width, height;
};
```

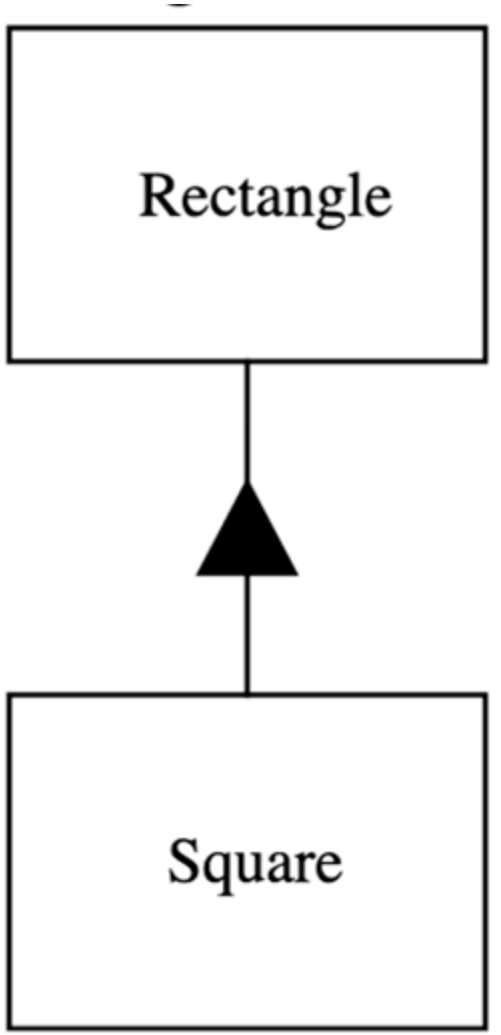


True? Square IS-A Rectangle?

Subtle LSP violation

```
struct Square: public Rectangle {
  void setWidth(const int w) {
    width = height = w; }
  void setHeight(const int h){ setWidth(h); }
};

Square s;
s.SetWidth(1); //OK
               // VVV breaks if r is Square
void f(Rectangle& r) { r.SetWidth(32); }
```



Breaks because `setWidth/setHeight` not virtual

LSP violation: Square is NOT a rectangle

```
...
  fix: virtual all the things
...
void g(Rectangle& r) {
  r.setWidth(2);
  r.setHeight(3);
//Open closed principle violated if r is Square
  assert(r.getWidth() * r.getHeight() == 6);
}
```

Only use virtual inheritance for pure IS-A relationships

Outside of UI, these relationships are rare!

In Java all methods are virtual by default

This means you should almost never use inheritance in Java

- Use `interface` instead
- Use if you are sure of IS-A relationship
- Use encapsulation for HAS-A relationships

```
interface IPart extends IDraw, IMore;
public class Whole implements IDraw {
    private IPart part;
    public void draw() {
        this.draw_background();
        part.draw();
    }
}
```

LSP: Use design by contract (Meyer)

Methods declare pre- and postconditions

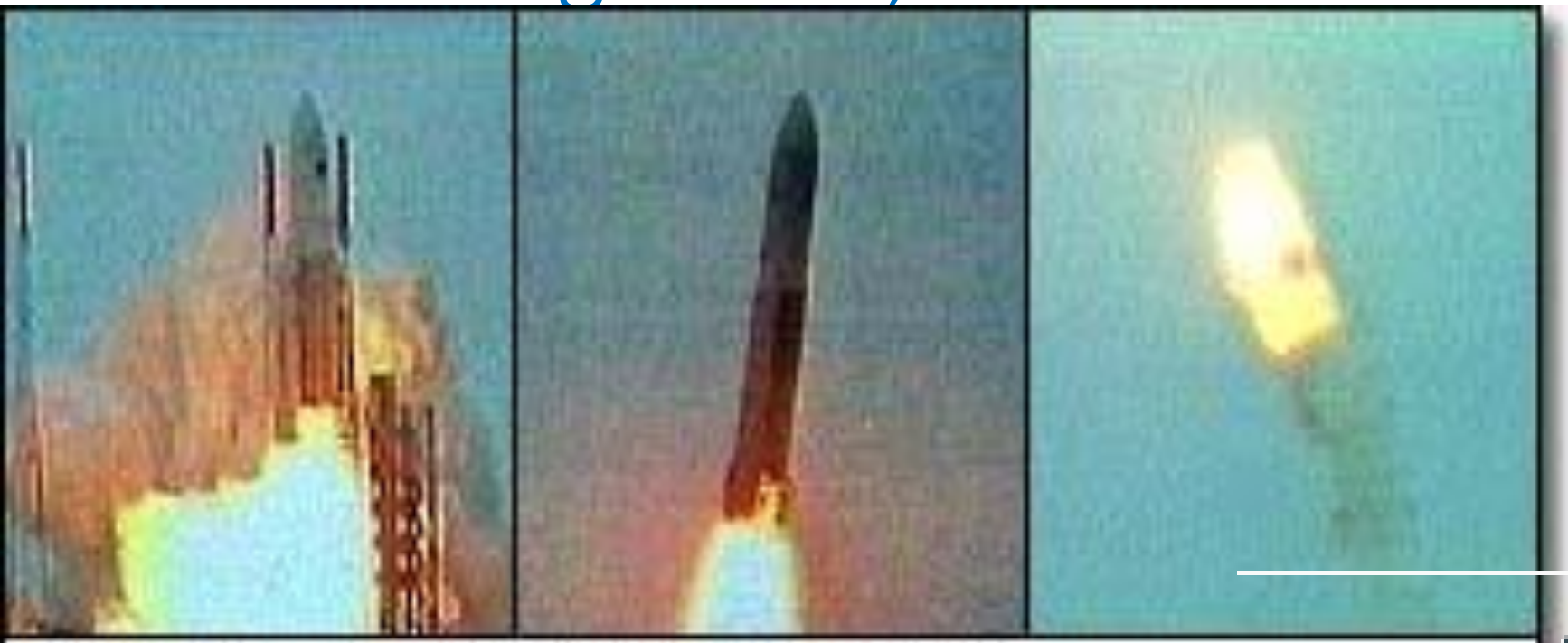
Example: C++26: contracts

```
bool canbe_normal(float3 v) { return v.x*v.y*v.z != 0.0; }

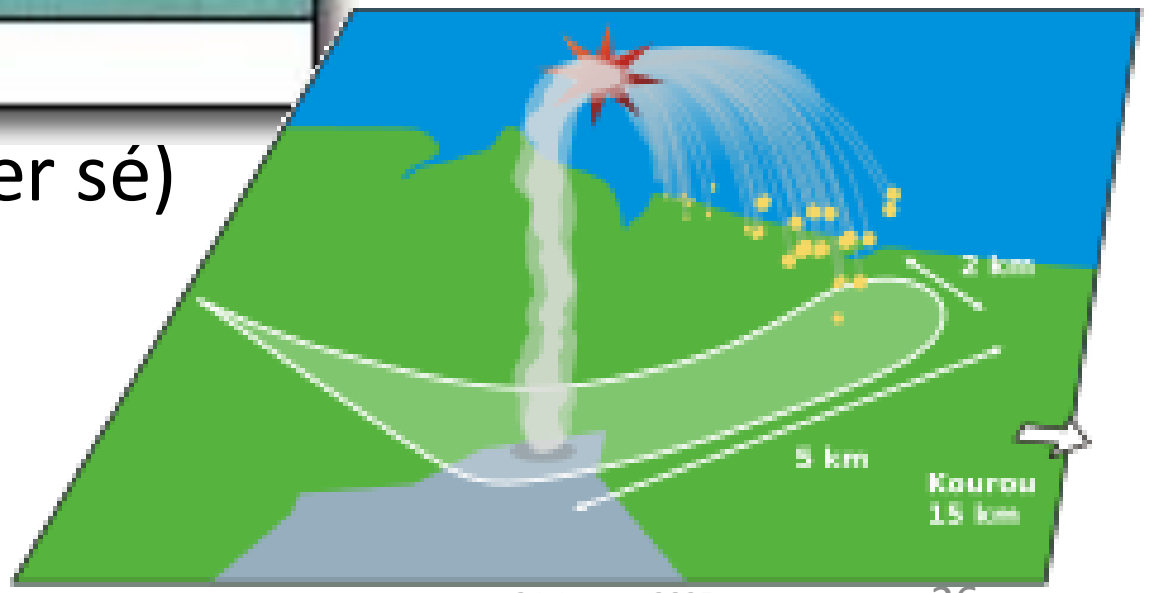
constexpr float3 normalize(float3 v) noexcept
pre(canbe_normal(v))//prevent division by 0, hence noexcept
post(v: is_normal(v)) {
  const auto norm = std::hypot(v.x, v.y, v.z);
  contract_assert(norm != 0.0);
  v.x /= norm; v.y /= norm; v.z /= norm;
  return v;
}
int main() {
  const auto v = normalize({0.3, 0.4, 0.5}); //succeeds
  const auto w = normalize({0, 0, 0}); //fails pre- and post
```

B. Meyer, *Object Oriented Software Construction*, Prentice Hall, 1988
https://en.cppreference.com/w/cpp/language/functions/function_contract_specifiers

Ariane flight V88, rocket 501



The rocket exploded seconds after launching



- Integer overflow (not a problem per sé)
- => Exception
- => Crash of guidance system

Check your pre-/postconditions

LSP: Use design by contract (Meyer)

Methods declare pre- and postconditions

Older languages: use assertions

```
bool canbe_normal(float3 v) { return v.x*v.y*v.z != 0.0; }
constexpr float3 normalize(float3 v) noexcept {
assert(canbe_normal(v))//prevent division by 0, hence noexcept
assert(v: is_normal(v)) {
  const auto norm = std::hypot(v.x, v.y, v.z);
  assert(norm != 0.0);
  v.x /= norm; v.y /= norm; v.z /= norm;
  return v;
}

int main() {
  const auto v = normalize({0.3, 0.4, 0.5}); //succeeds
  const auto w = normalize({0, 0, 0}); //fails pre- and post
}
```

B. Meyer, *Object Oriented Software Construction*, Prentice Hall, 1988
https://en.cppreference.com/w/cpp/language/functions/function_contract_specifiers

Explicit contracts vs asserts vs errors

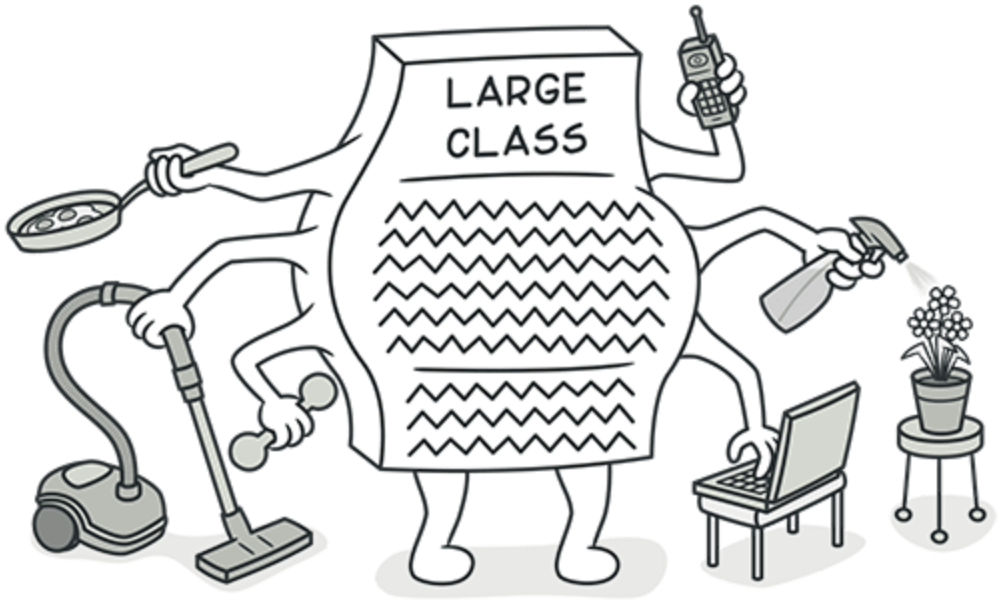
- Contracts (pre/post/middle conditions) => tooling support
- contract violations cannot be recovered from
  - errors can be accommodated in code => failback options

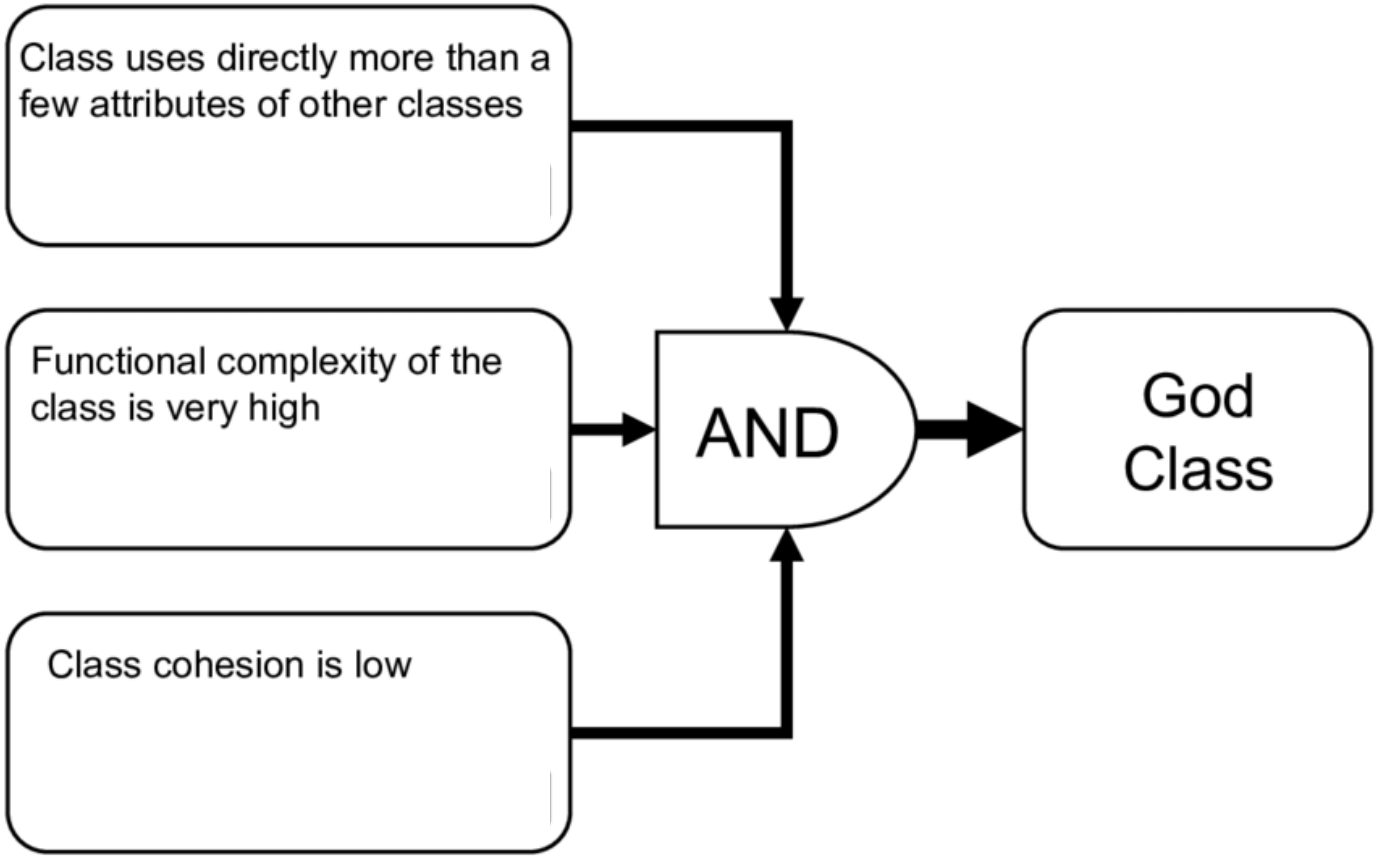
- pre: input parameters + pre state of object
- post: output + post state of object
- invariants: assertions in the middle

- In actual code, contract assertions can only catch subset of violations, cannot prove full program correctness
  - program correctness is undecidable [Turing]
  - language features not yet comprehensive

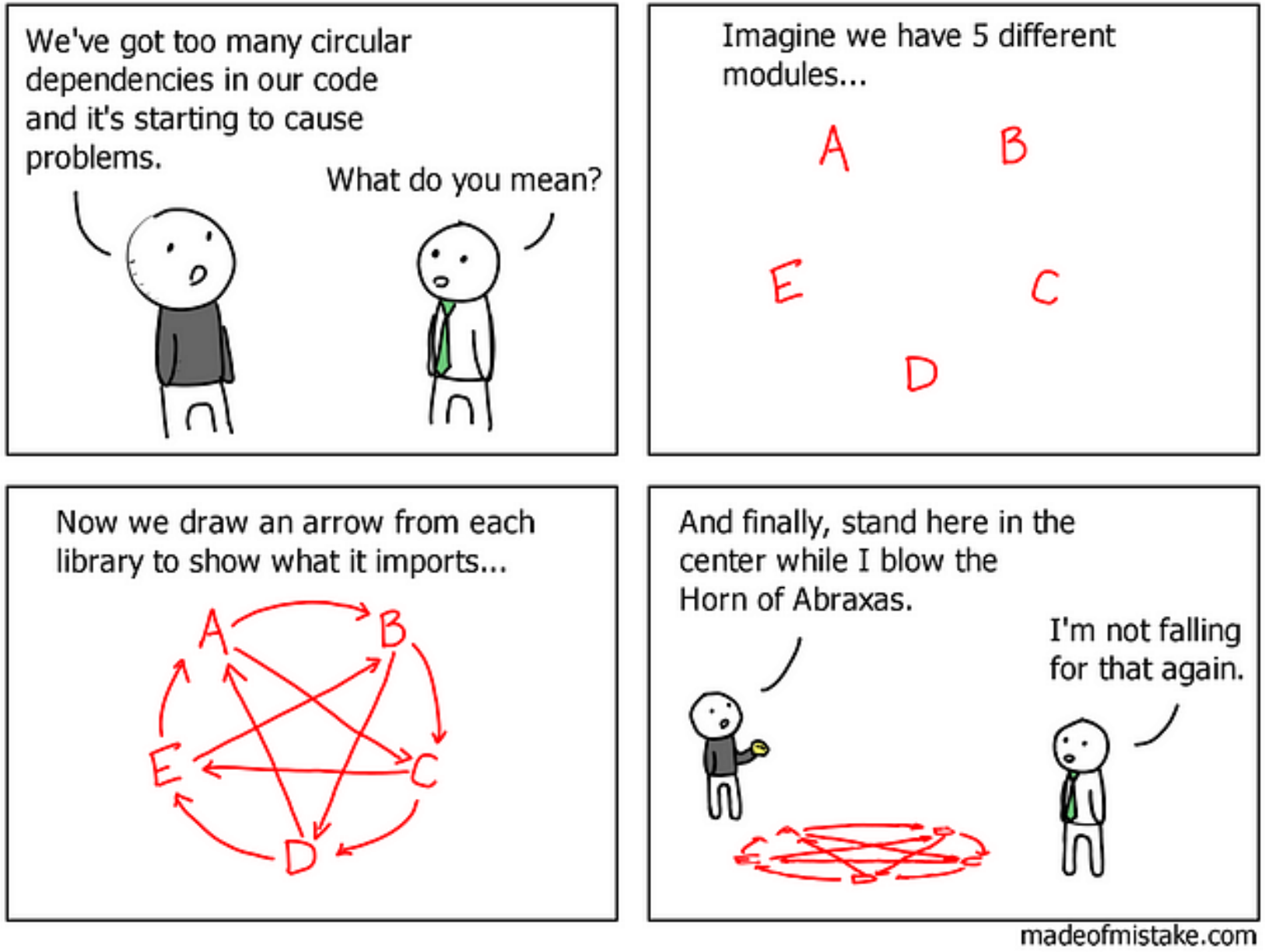
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2900r14.pdf
M. Davis (ed.), The Undecidable, Basic Papers on Undecidable Propositions, Unsolvable Problems And Computable Functions, 1965 (Paper #3)

No god classes





CSC3003S ASD11
SOLID design principles (OOP)



- **S**ingle Responsibility
- **O**pen Closed
- **L**iskov substitution
- **I**nterface Segregation
- **D**ependency Inversion

Interface segregation

Don’t use fat interfaces

“Clients should not be forced to depend upon (parts of) interfaces that they do not use.”

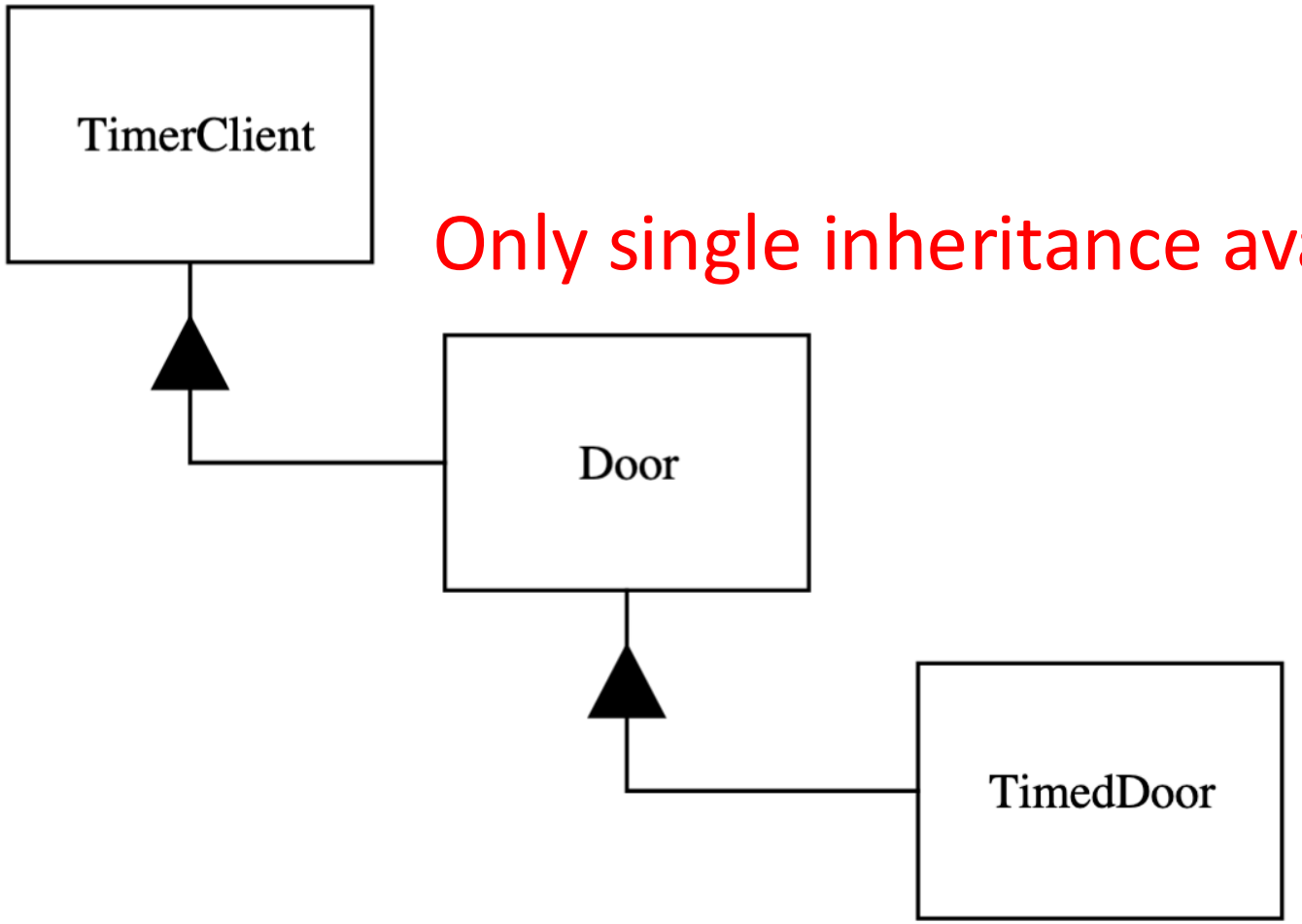
Interface segregation

Don’t use fat interfaces

```
public interface IDoor {
    public void lock();
    public void unlock();
    public bool isDoorOpen();
}
public interface ITimerClient {
    public void TimeOut();
}
public class Timer {
public:
    void register(int timeout, TimerClient client);
}
```

ISP: How to make a timed door?

```
public class Door implements IDoor, ITimerClient;
public class TimedDoor extends Door;
```



Only single inheritance available

ISP: How to make a timed door?

```
public class Door implements IDoor, ITimerClient;
public class TimedDoor extends Door;
```

What happens when `ITimerClient` changes?

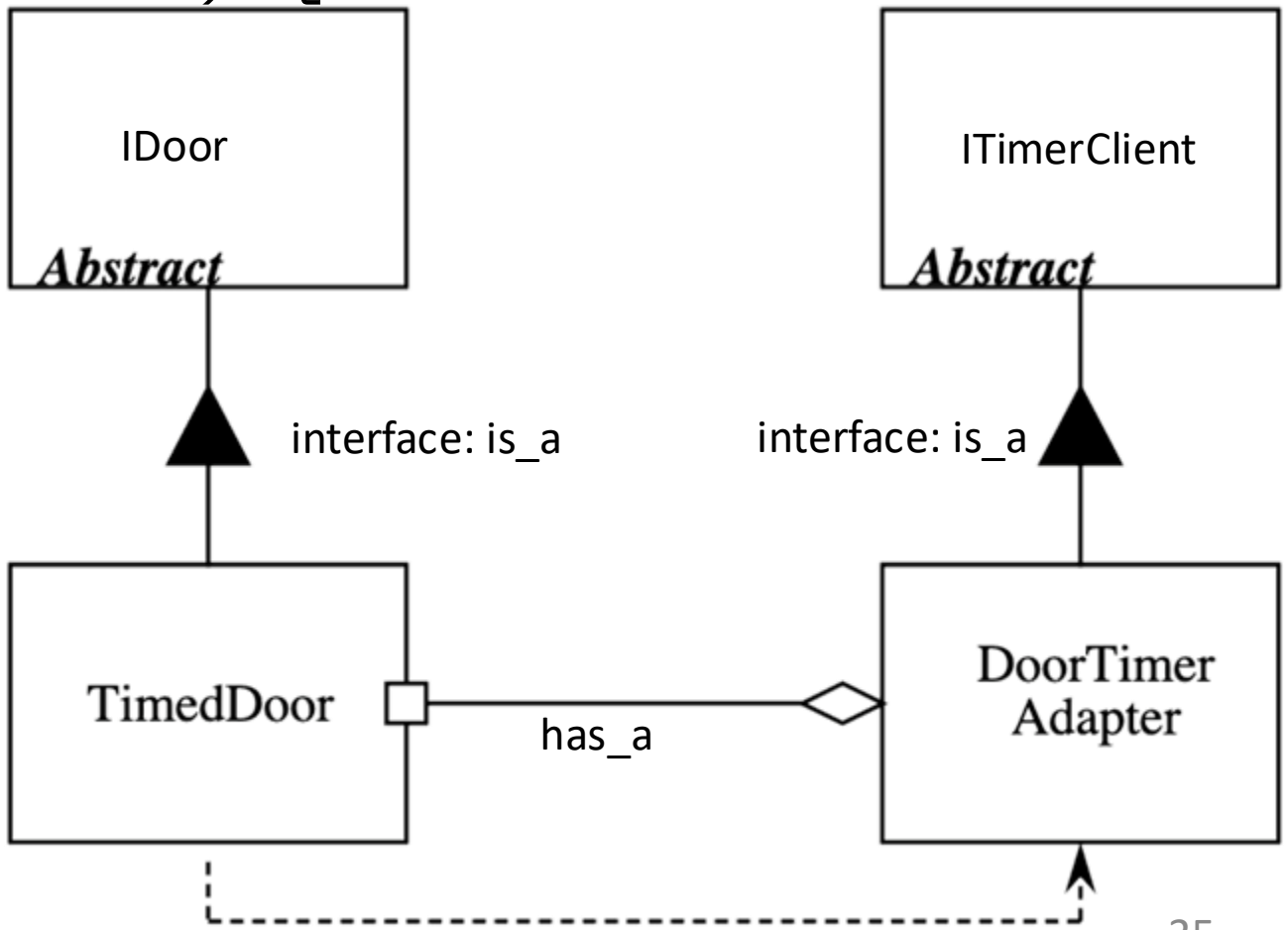
=> Door needs to change as well, even though it does not use a timer.

Solution: separate using delegation

```
public class TimerDoor implements IDoor {
   public DTAdaptor dta;
   public void regTimer(int end) {
      Time t = toTime(end);
      dta.register(t);
   }
}
```

Only single inheritance available

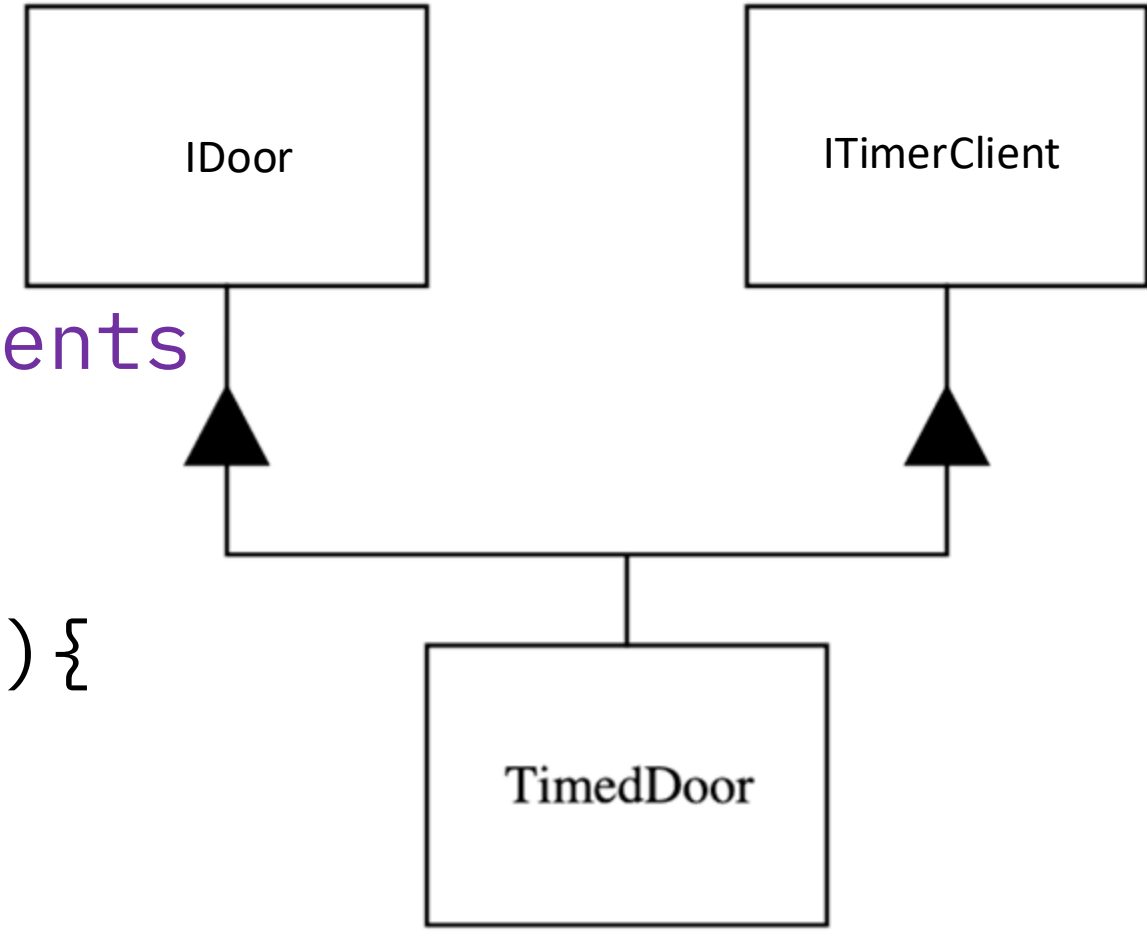
Workaround using the Adapter pattern



If interfaces, implement multiple intf.

Using interfaces, you'll still need to implement or embed the relevant objects

```
public class TimerDoor implements
IDoor, ITimerClient {
   public DTAdaptor dta;
   public void register(Time t){
      dta.register(t);
   }
}
```



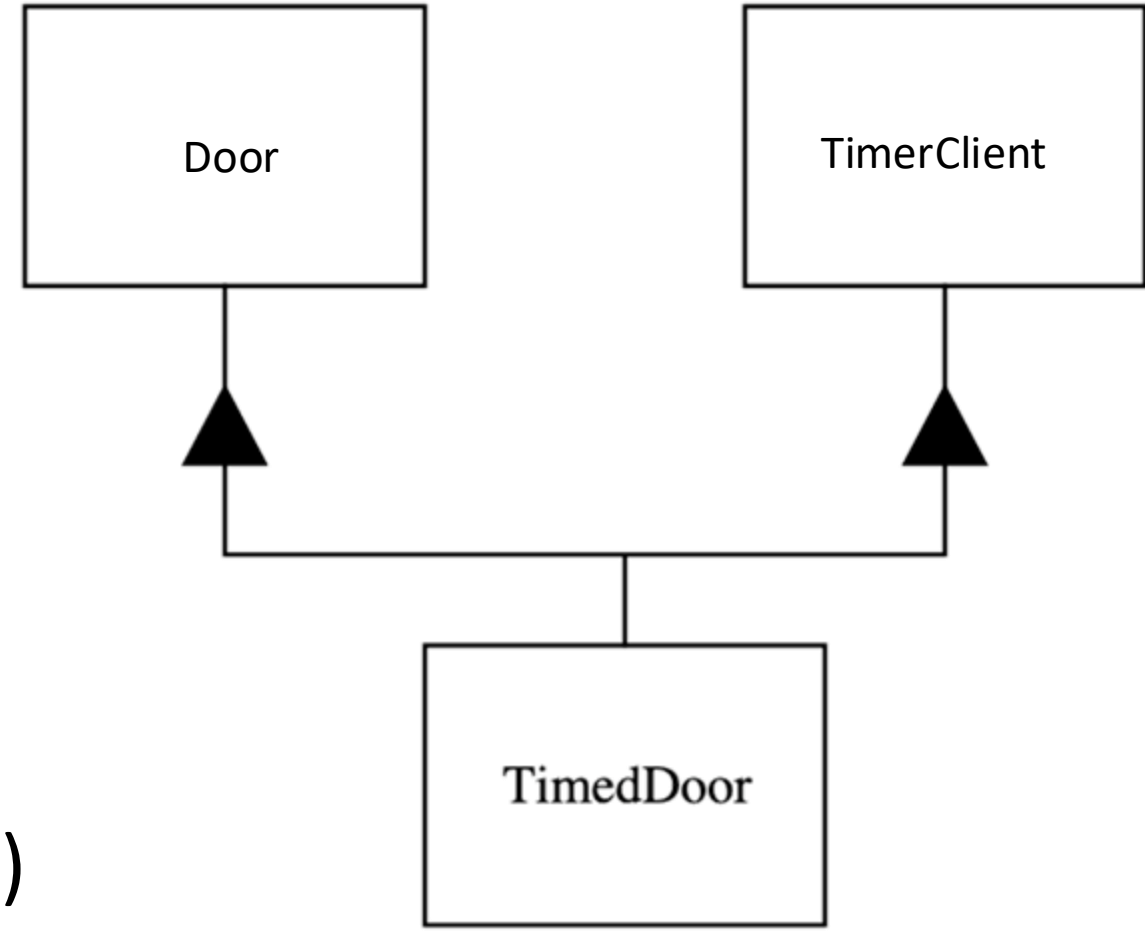
C++ multiple inheritance

Using interfaces, you'll still need to implement or embed the relevant objects

```
class TimerDoor: public Door,
public TimerClient {
  //inherits register method
}
```

Rarely used, because complex.

(except for abstract classes == interfaces)

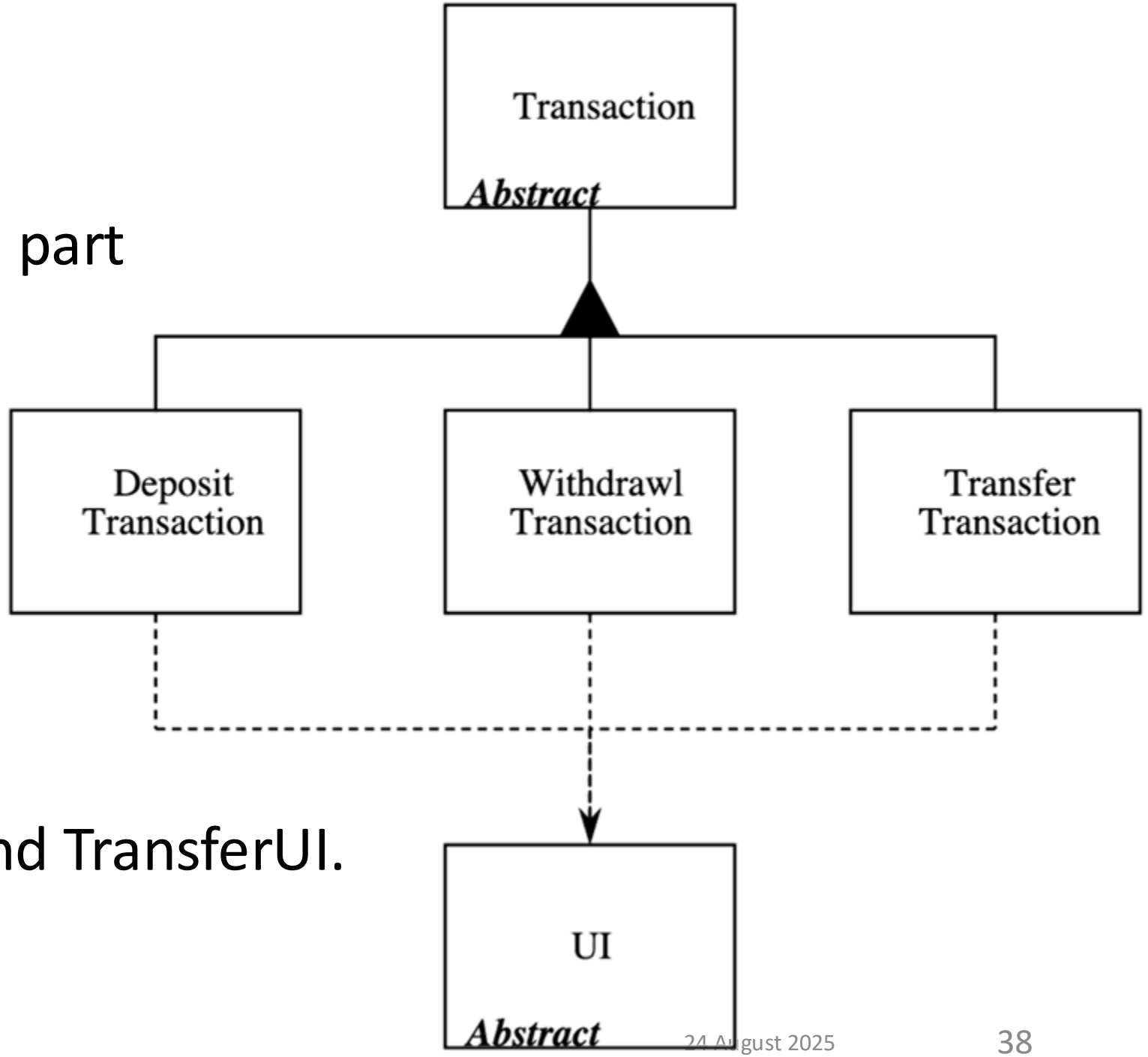


ATM example

Different transactions

Bad, because each uses a part

no other UI needs.



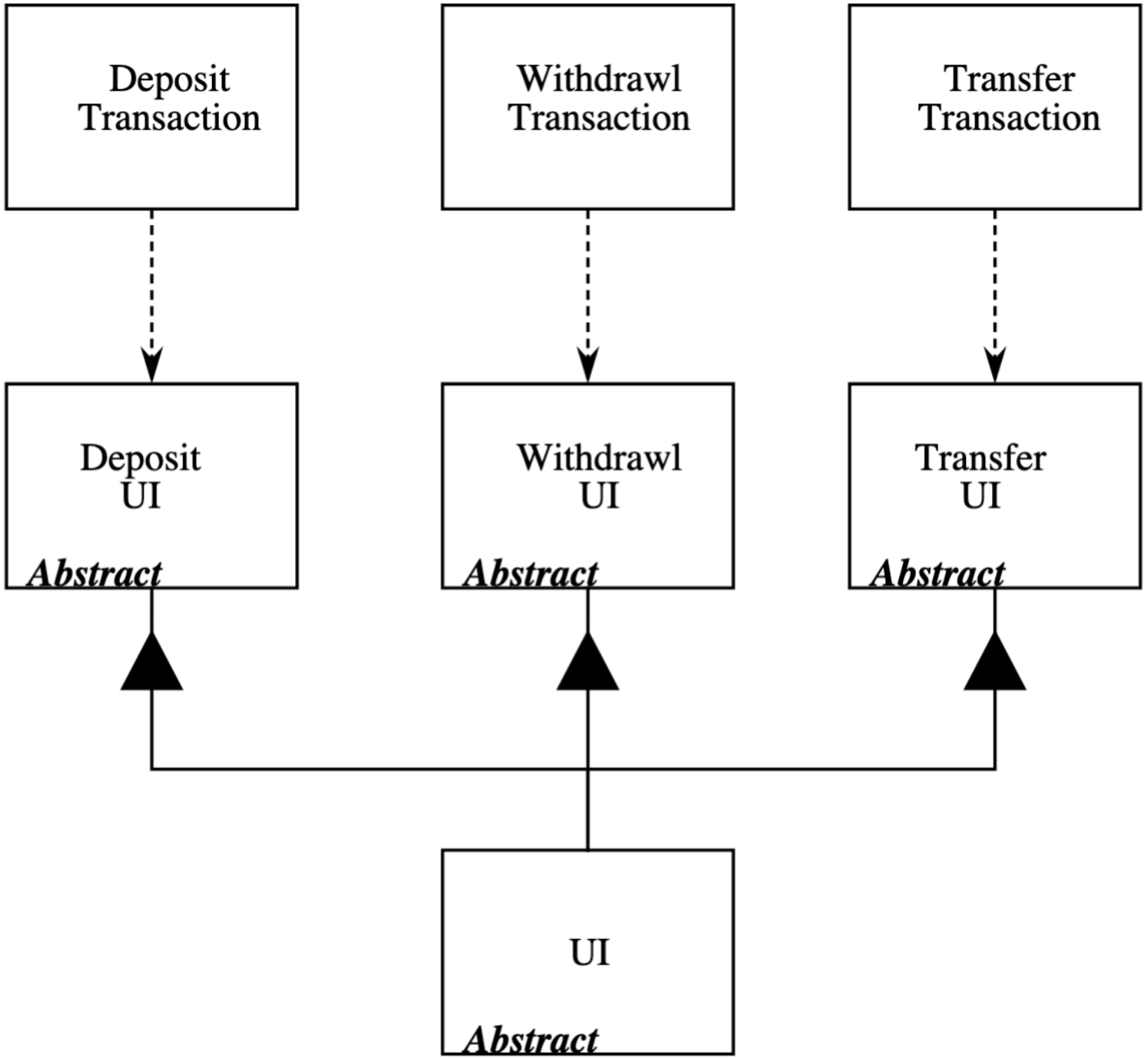
Split UI into:
DepositUI, WithdrawUI and TransferUI.

ATM example

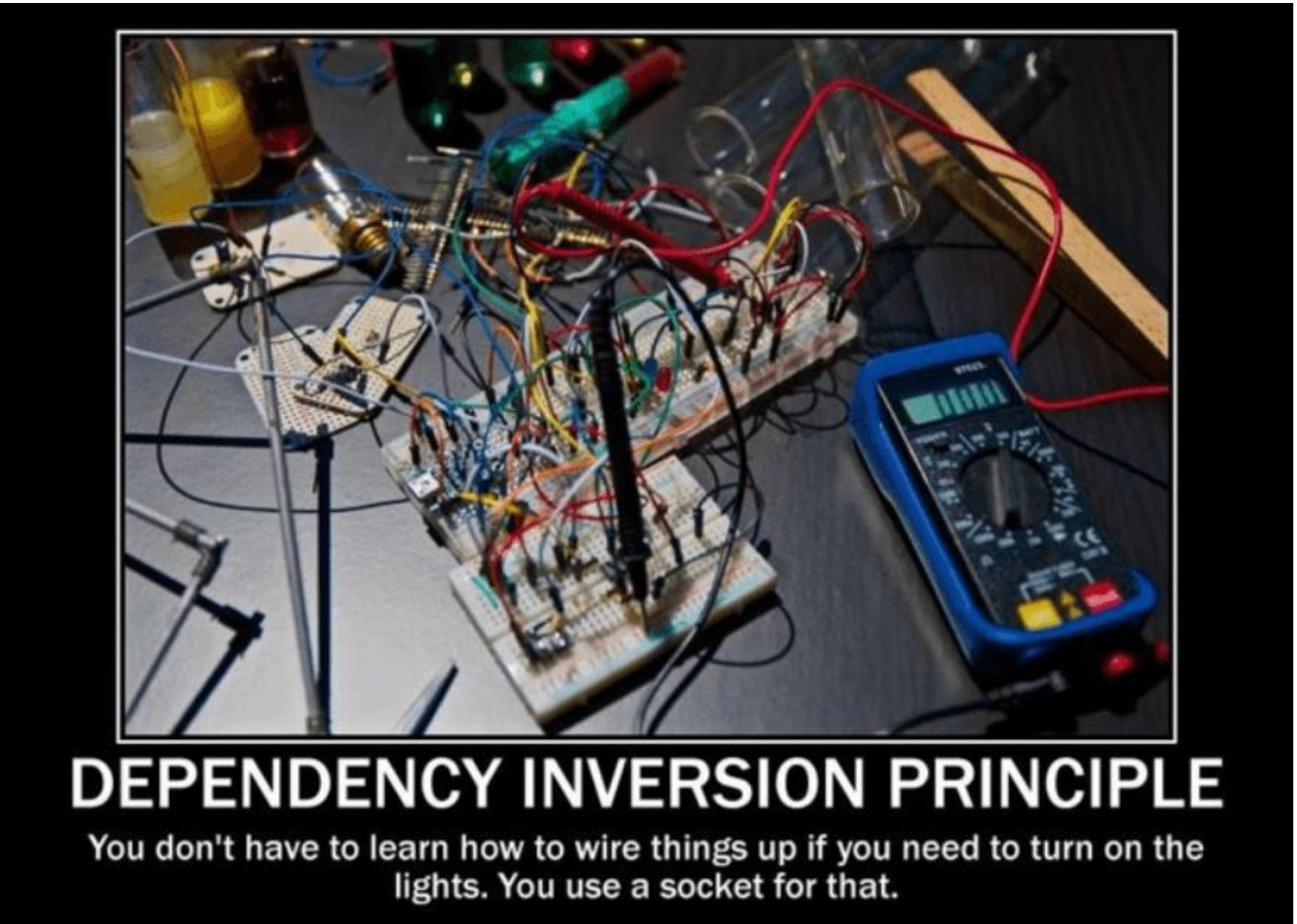
Each UI only has code supporting that transaction

UIs internally still use overall UI.

Allows decoupling of transactions into separate modules.
Errors in deposit don't lead to accidental withdrawals.



CSC3003S ASD11
SOLID Design principles (OOP)



Single Responsibility

Open Closed

Liskov substitution

Interface Segregation

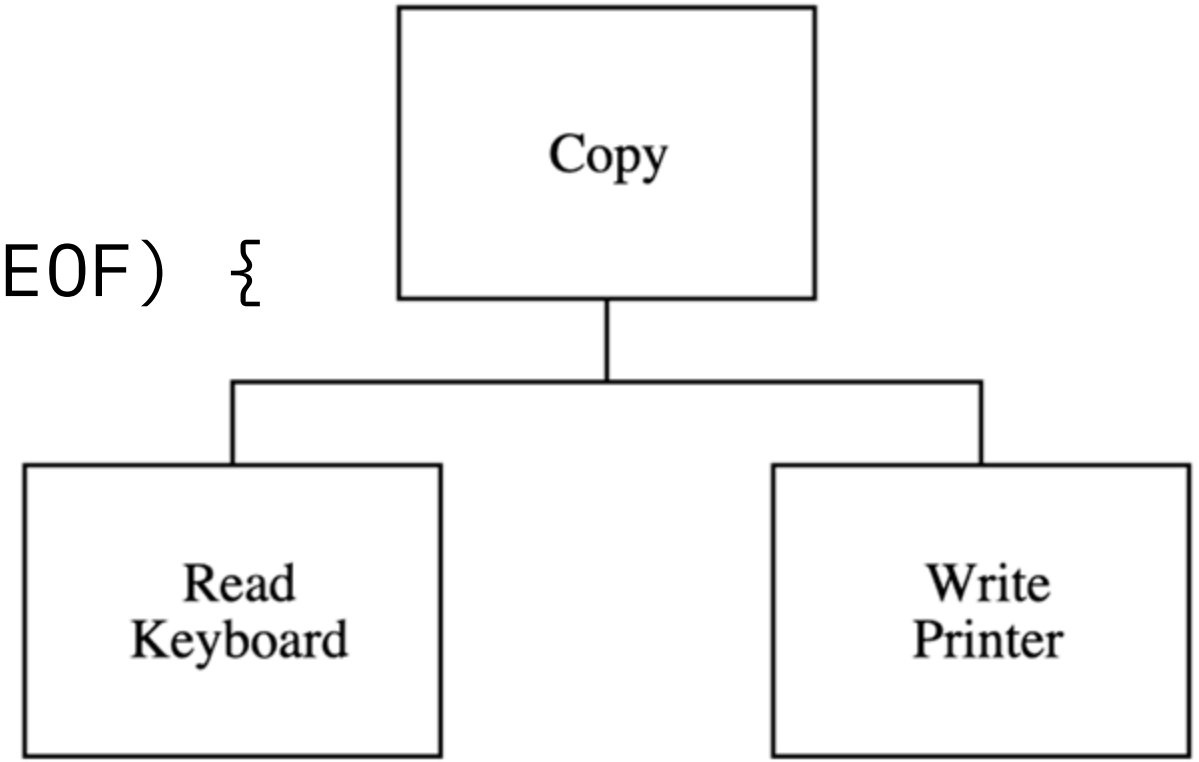
Dependency Inversion

Dependency inversion

A. High level modules should not depend upon low level modules.
Both should depend upon abstractions.

B. Abstractions should not depend upon details.
Details should depend upon abstractions.

```
//bad code
void Copy() {
   char c;
   while ((c = readKey()) != EOF) {
      print(c);
   }
}
```



Dependency inversion

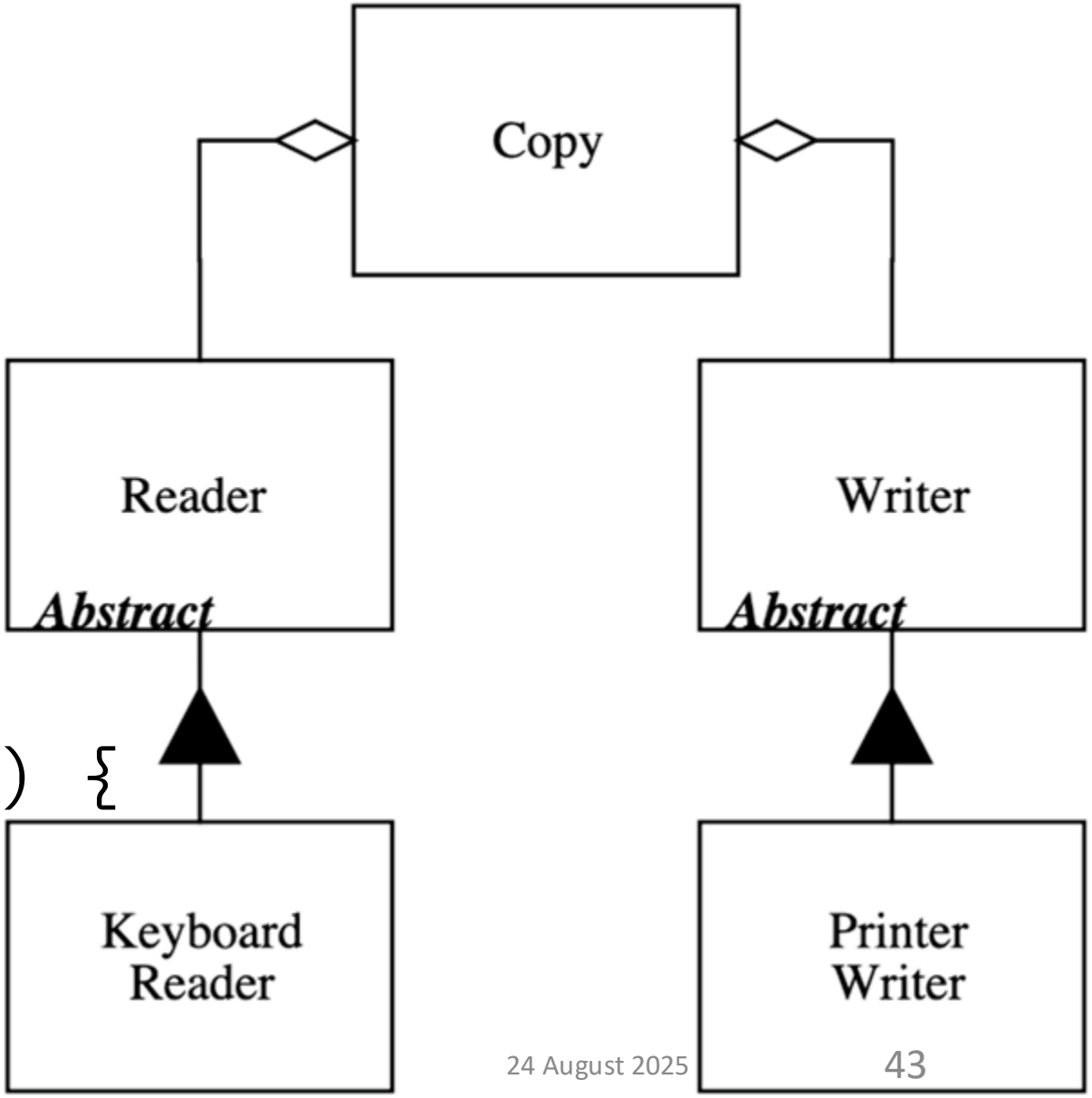
Problem: Copy method not reusable for other contexts

```
//bad fix
enum OutputDevice { printer, disk }; //moar devices
void Copy(outputDevice dev) {
  int c;
  while ((c = ReadKeyboard()) != EOF) {
    if (dev == printer) { WritePrinter(c); }
    else { WriteDisk(c); }
  }
}
```

Support any device

Solution: Make copy method use readers & writers

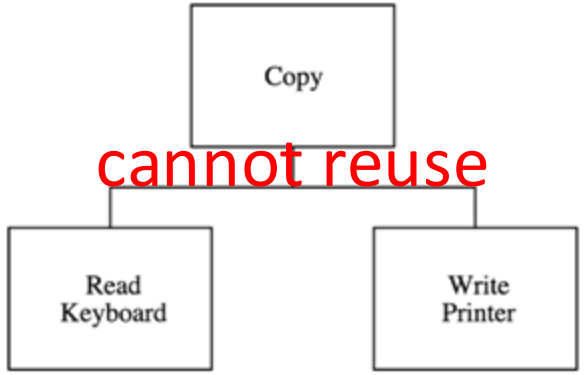
```
interface Reader {
   char Read();
}
interface Writer {
   void Write(char c);
}
void Copy(const Reader& r,
          Writer& w) {
   char c;
   while((c=r.Read()) != EOF) {
      w.Write(c);
   }
}
```



Why “inversion”?

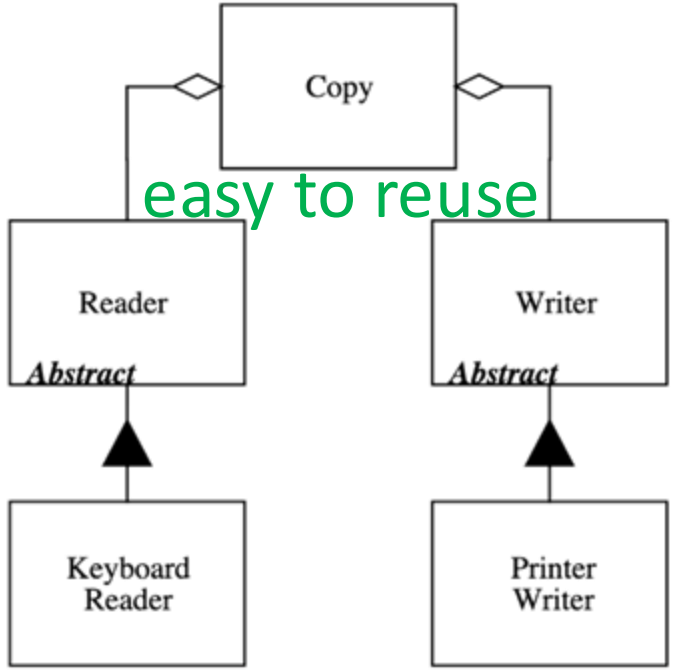
Traditional structured design (1990’s):

- High level modules depend on low level modules
- Abstraction depends on details



But:

- High level modules contain logic and policy
- Low level changes must not break higher levels
- We want to reuse high level modules
- Use abstractions to hide low level details



Fin

“All problems in computer science can be solved by another layer of indirection.

Except the problem of too many layers of indirection.”

-- Butler Lampson

..., but that usually will create another problem.”

-- David Wheeler